Understanding Sessions in Agent to Agent Communication

Posted **Sep 27, 2025**

By **Christian Posta**

6 min read

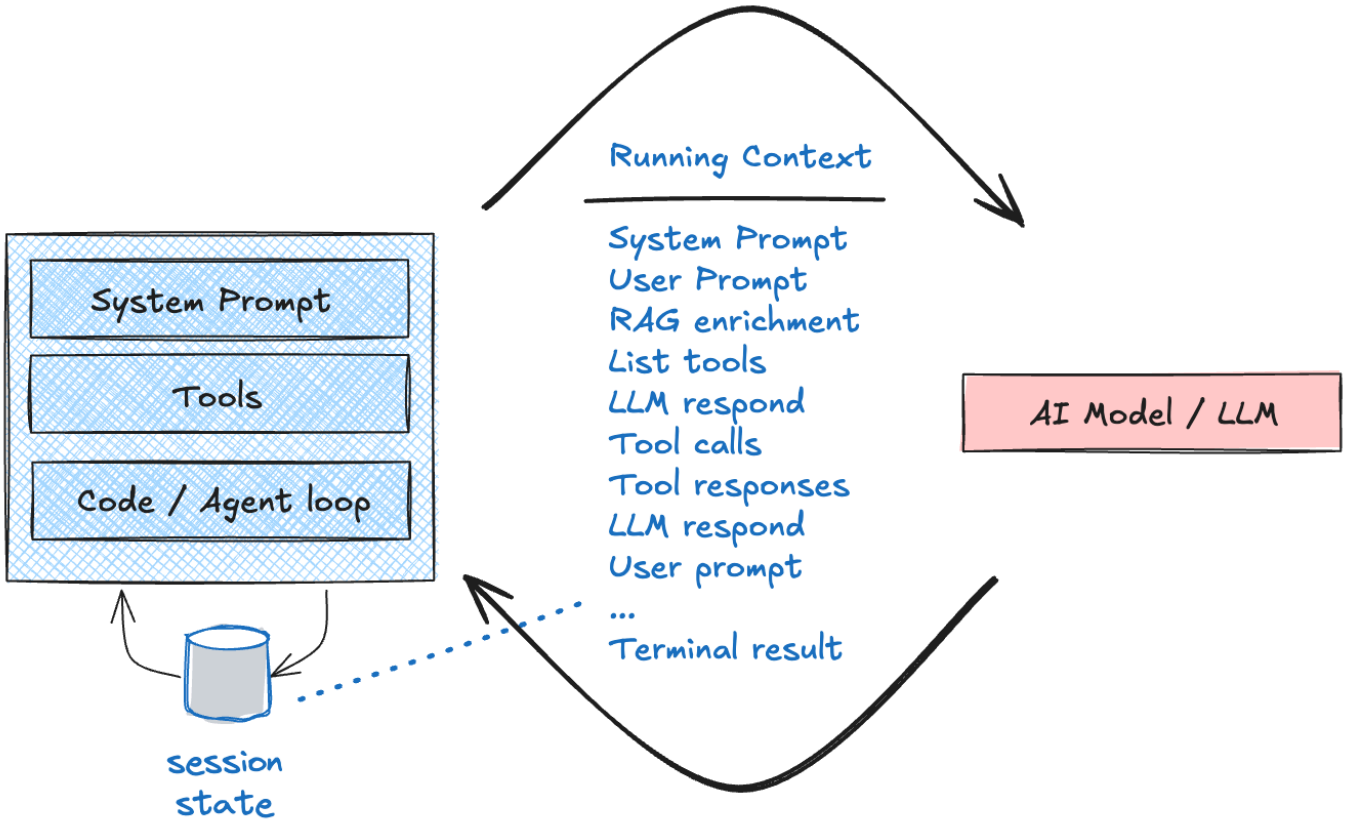
Contents >

The more we dig into enterprise usecases around Agent to Agent (A2A) and Agent to MCP scenarios (MCP), the more questions pop up that I'm interested in discussing and sharing. In this post we'll take a look at "sessions" in agents (and MCP). Follow (@christianposta or /in/ceposta) for these types of blogs.

This can be a complex topic, but I'll try to distill it. When something triggers an agent to perform an action (ie, a user prompts the agent, or the agent reacts to an observed event), it may perform some actions autonomously to achieve its goal. This may involve reaching out to other agents or tools (via MCP). In an enterprise setting, it's crucial to be able to track causality and decision points for audit, compliance, performance, and security. I've discussed in the past the need for agent delegation vs impersonation, but this post looks at it from a session standpoint.

Agent State

Agents are stateful constructs. An agent's main purpose is to manage the context it builds to interact with the LLM. An agent starts off with a set of instructions (system prompt), a set of tools, and an AI model (LLM) that they connect with. They take those pieces, a user / event prompt, and then begin building the context and conversing with the LLM. It may perform a RAG lookup, call out to tools to get more data, track the LLM's responses, take more user input, etc. All of this is captured in the context.



An agent make take a while to work toward a goal. It may involve multiple turns with the calling client (user, event, another agent, etc). This context needs to be associated with the caller. In the A2A protocol, the context is associated with a `contextId`. When a client connects up to the agent for the first time, the agent will respond with a payload that includes this `contextId`. This acts as the identifier that maps to the context for that client. Each time the client interacts with the agent, it will send this `contextId`.

Client message:

```json
```

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "message/send",
  "params": {
    "message": {
      "role": "user",
      "parts": [
        {
          "kind": "text",
          "text": "tell me a joke"
        }
      ],
      "messageId": "9229e770-767c-417b-a0b0-f0741243c589"
    },
    "metadata": {}
  }
}
```

Agent responds:

JSON

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "result": {
    "messageId": "363422be-b0f9-4692-a24d-278670e7c7f1",
    "contextId": "c295ea44-7543-4f78-b524-7a38915ad6e4",
    "parts": [
      {
        "kind": "text",
        "text": "Why did the chicken cross the road? To get to the other side!"
      }
    ],
    "kind": "message",
    "role": "agent"
  }
}
```

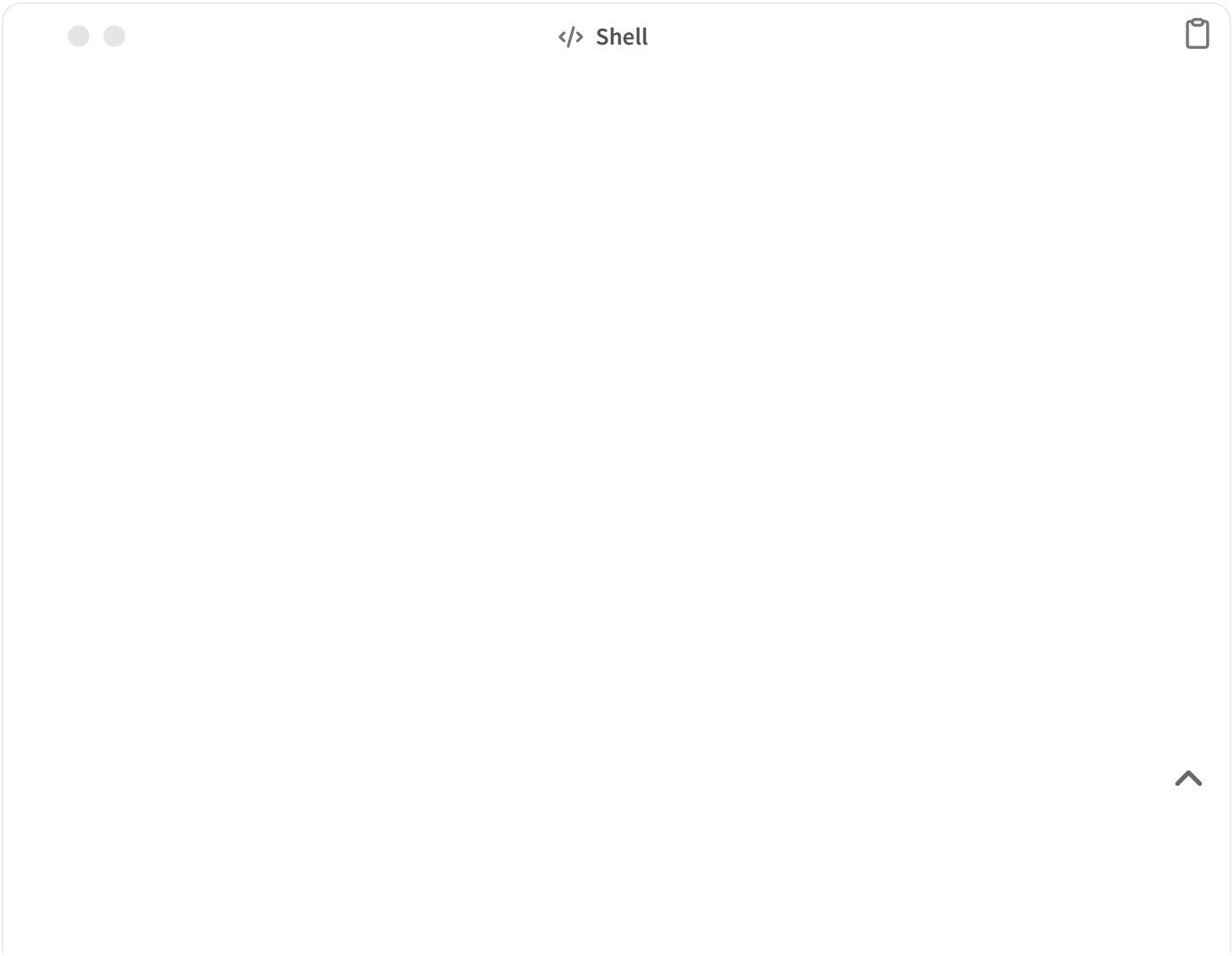
Note the `"contextId": "c295ea44-7543-4f78-b524-7a38915ad6e4"` . Any further interactions from the client should use this contextId. The agent maps up the client's interaction with the correct context with this `contextId` .

NOTE: A2A also specifies a `taskId` which can be used to track specific, more granular tasks within a single context. You can think of the `contextId` as the overall state identifier and specific `taskId` s as sub tasks within the overall context. We won't say too much more about `taskId` in this blog. Feel free to reach out with questions if there is something that is unclear.

MCP Server State

If your agent interacts with tools via the Model Context Protocol (MCP), there is another type of session to be aware of. In MCP, servers can also keep state and may issue a way to track that. As the name suggests "Model Context" is the primary type of state kept in an MCP server. It can track tool calls, responses from tools, combine results from previous tool calls, expose tool calls (or general state) as Resources, request model sampling from LLMs, etc. Over multiple interactions from a client to the MCP server, this state/context can evolve. The MCP server communicates an identifier called the `Mcp-Session-Id` in a response HTTP header. The client then uses this session-id in all future communication with the MCP server.

MCP client / agent sends:



```
POST /mcp HTTP/1.1
Host: mcp-server.solo.io
Content-Type: application/json
MCP-Protocol-Version: 2025-06-18

{
  "jsonrpc": "2.0",
  "id": "init-1",
  "method": "initialize",
  "params": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "roots": {
        "listChanged": true
      },
      "sampling": {}
    },
    "clientInfo": {
      "name": "MyMCPClient",
      "version": "1.0.0"
    }
  }
}
```

MCP Server responds with:

```
Shell
```

```
HTTP/1.1 200 OK
Content-Type: application/json
Mcp-Session-Id: 1868a90c-4c2f-4a5b-8d3e-7f2a6c1b9e8d

{
  "jsonrpc": "2.0",
  "id": "init-1",
  "result": {
    "protocolVersion": "2025-06-18",
    "capabilities": {
      "resources": {
        "subscribe": true,
        "listChanged": true
      },
      "tools": {
        "listChanged": true
      },
      "logging": {}
    },
    "serverInfo": {
      "name": "CustomerServiceMCP",
      "version": "2.1.0"
    }
  }
}
```

Note the HTTP Header in the response `Mcp-Session-Id: 1868a90c-4c2f-4a5b-8d3e-7f2a6c1b9e8d`.

The client sends this in future requests:

Shell

```
POST /mcp HTTP/1.1
Host: mcp-server.example.com
Content-Type: application/json
MCP-Protocol-Version: 2025-06-18
Mcp-Session-Id: 1868a90c-4c2f-4a5b-8d3e-7f2a6c1b9e8d

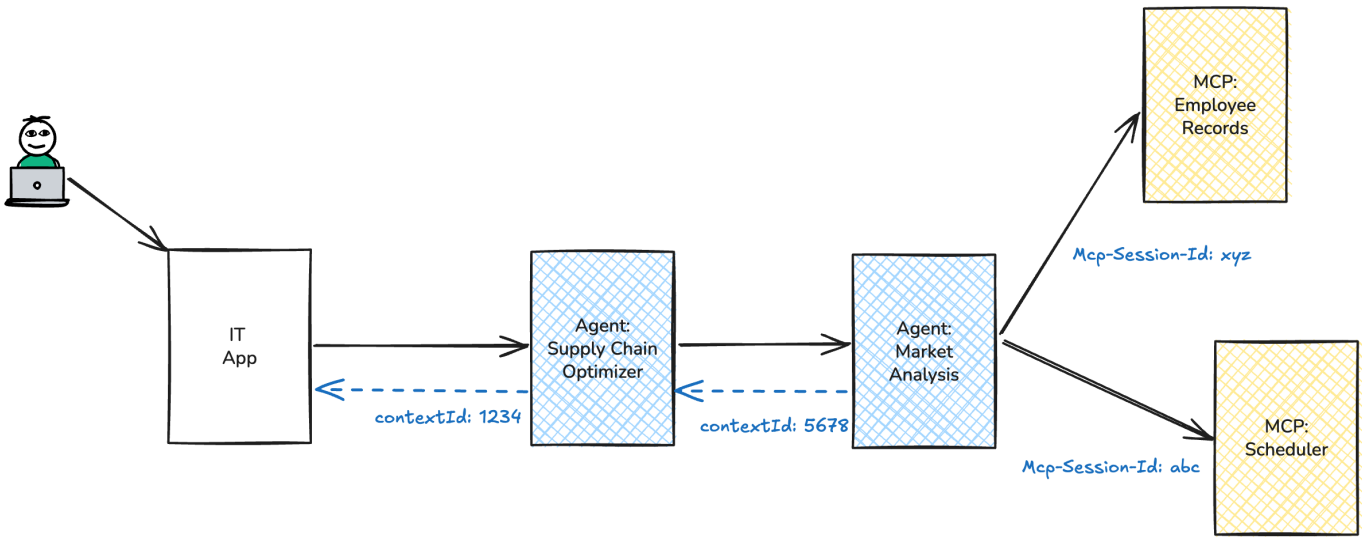
{
  "jsonrpc": "2.0",
  "method": "notifications/initialized"
}
```

Bigger picture

To recap:

- each agent can keep context/state over a series of interactions from a user/event tracked as a `contextId`
- each MCP server can keep context/state over a series of interactions from a client, tracked as an HTTP Header `Mcp-Session-Id`

What does this look like across multiple agents, and multiple MCP servers?



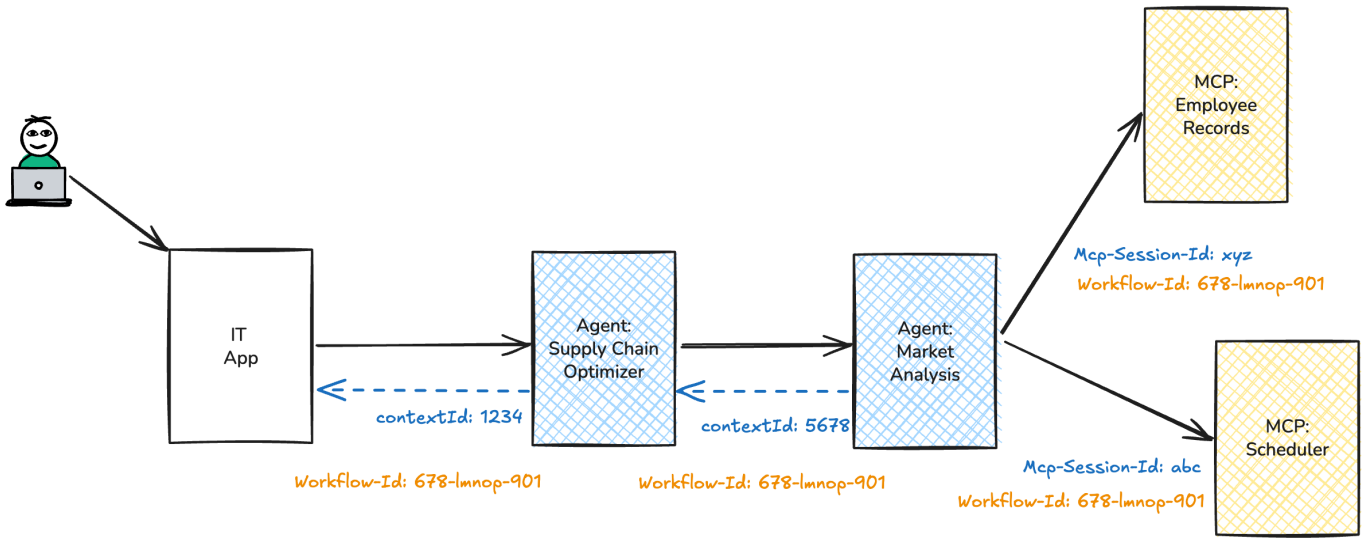
You can see that an agent keeps track of it's context for a particular interaction. An MCP server also keeps track of its state for a particular interaction. But we need a way to tie these together to track causality: e.g., session `abc` in the Scheduler MCP server has a relationship to the context `1234` in the supply chain agent, and all of this is related to the original execution/workflow initiated by the user. Neither the A2A or MCP protocols really specify these relationships, but that's okay. An organization can establish it's own "workflow id" the same way it probably already does.

Tying together with a Workflow Id

We need a way to tie all of these pieces of state together. We have a few different options here.

- Establish a convention around a `Workflow-Id` that all agents / MCP servers honor
- Piggy back on top of any tracing already in place (ie, OTEL trace Id)
- Tie into any existing mechanisms around a long-running "transaction"

For example, our previous diagram could be drawn with a common `Workflow-Id: Workflow-Id: 678-lmnop-901`



An incredibly important part of this workflow, additionally, is how you tie it into the authorization context. That is all of these sessions (agent context / mcp session) must be tied to an user/agent's/mcp server's authorization context. We want both:

- disallow clients attaching to context/state they are not authorized to see
- cryptographically track the workflow across multiple agents/servers/APIs/transports

This blog is already getting long, so I'll leave that for a separate post. Follow (@christianposta or /in/ceposta) for more.

AI Agents

agents sessions

This post is licensed under **CC BY 4.0** by the author.

Share:

Further Reading

May 1, 2018

My Sessions at Red Hat Summit in San Francisco

I’m fortunate to have a few sessions at Red Hat Summit next week (May 8-10) in San Francisco! I’ll be speaking/co-presenting with some fantastic presenters about microservices, service mesh, Istio,...

May 6, 2025

Part II - Mitigating MCP and A2A Attack Vectors for AI Agents

Mitigations Potential Mitigations for Naming Attacks To address these vulnerabilities, we should consider: Strict Registration Policies: Implementing formal registration systems with ver...

Apr 30, 2025

Deep Dive MCP and A2A Attack Vectors for AI Agents

The Model Context Protocol (MCP) and Agent 2 Agent (A2A) specification are similar RPC style protocols that specify interaction between Agents and Tools (MCP) and Agents and other Agents (A2A). The...

OLDER
Can You Use an API Gateway as an MCP Gateway?

NEWER
Building an MCP Gateway with Apigee API Gateway

An error occurred: giscus is not installed on this repository

© 2026 **Christian Posta**. Some rights reserved.
Using the **Chirpy** theme for **Jekyll**.